

LAB 6

La scena completa appare in questo modo.



1. Skybox

Per realizzare un effetto di immersione ambientale, sono stati utilizzati gli shader *CubeMap.glsl* (vertex e fragment shader). Questi shader permettono di applicare sei texture diverse alle facce di un cubo, creando l'illusione di un **ambiente tridimensionale** che circonda completamente l'osservatore. Nel vertex shader, le coordinate dei vertici del cubo vengono trasformate **senza**

applicare la traslazione della vista, così la skybox appare fissa rispetto alla telecamera anche quando ci si muove nello spazio. Nel fragment shader, viene campionata la texture cubica (cubemap) usando direttamente la direzione del vertice come vettore. È stato rimosso il piano ondulado che simulava il mare.

2. Texture mapping 2D

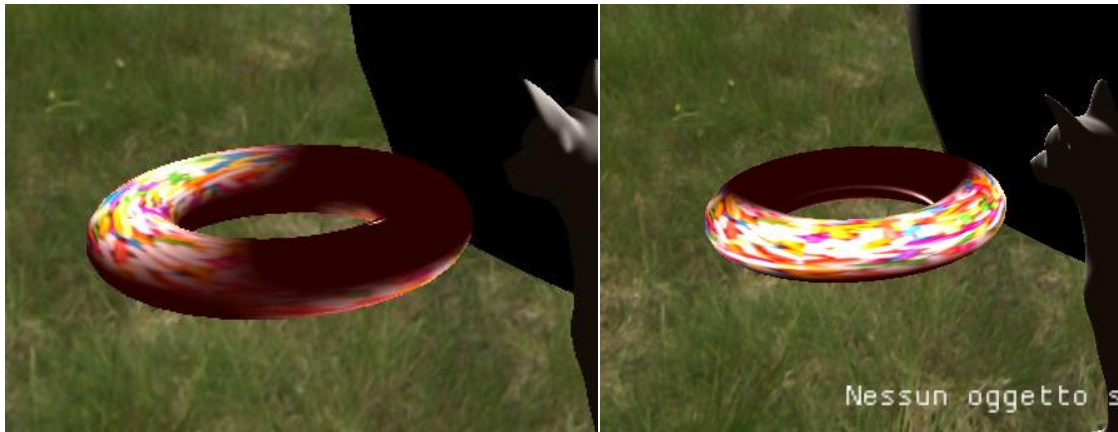
Nel fragment shader, è stata introdotta la gestione della texture per determinare il colore finale del pixel. L'immagine (nomefile.jpg) viene passata allo shader dall'applicazione. Per applicare correttamente la texture sulla superficie del toro, è stato necessario associare a ogni vertice delle **coordinate texture (UV)**, operazione eseguita nel file *init_geometrie*. L'idea è quella di mappare l'intera superficie come se la texture si arrotolasse su di essa. In questo caso l'illuminazione dell'oggetto non è realistica.



3. Texture mapping 2D + Shading

Per integrare l'illuminazione Phong alla visualizzazione con texture, ho modificato il **vertex shader** in modo da calcolare e passare al fragment shader alcune variabili essenziali: la posizione del vertice nello spazio camera (*vcsPosition*), la posizione della luce nello spazio

camera (`vcsLightPosition`) e la normale nello spazio camera (`vcsN`). Nel **fragment shader**, il colore del pixel viene prelevato dalla texture e poi combinato con le componenti di illuminazione ambientale, diffusa e speculare del modello Phong. Questa combinazione permette di ottenere un'illuminazione più realistica sulla superficie texturizzata.



4. Reflection

La **reflection** permette di riflettere sull'oggetto la scena circostante (in questo caso è applicato alla sfera), in questo caso la texture inserita nella skybox. Nel fragment shader calcoliamo il **vettore riflesso** e lo utilizziamo per effettuare un campionamento sulla **cubemap**. In questo shader riscontro però un problema: la texture è ruotata di 90 gradi. Ho provato a cambiare il segno di R ma il problema non si è risolto e anche provando ad eseguire il codice su un'altra macchina il problema non si è risolto.



5. Refraction

La **refraction** permette di realizzare un oggetto trasparente consentendo di vedere ciò che si trova dietro di esso. Nel processo di rifrazione, la direzione della vista viene leggermente deviata quando il raggio entra nell'oggetto, e questa nuova direzione viene usata per campionare la cubemap. Nel fragment shader infatti calcoliamo un grado di rifrazione che usiamo per determinare la direzione del raggio rifratto che poi usiamo per effettuare il campionamento sulla



cubemap. In questo caso, però, sembra ci sia qualche problema con le normali: osservando il cubo si notano chiaramente i triangoli che costituiscono le sue facce. Anche invertendo i segni o eseguendo il codice su un altro computer, il risultato non cambia.

6. Procedural texture mapping

Come **texture procedurale** ho realizzato un'altra bandiera, cioè quella del Giappone. La texture procedurale è realizzata dal metodo *loadMyTexture_proc()*. Nella funzione calcoliamo la distanza (distanza euclidea senza l'utilizzo della radice quadrata, quindi avremo la distanza al quadrato) di ogni punto dal centro e se la distanza è all'interno del raggio definito allora coloro il pixel di rosso altrimenti lo coloro di bianco.

